A Map of Models

Unit 2: What Is on the Menu, and How to Choose

Stat 220 | Statistics for Data Science

Where we left off

- Unit 1 compared two groups. One estimate, one standard error, one question.
- Almost nothing you will be handed at work looks like that. You get a table with forty columns and a vague question attached.
- This unit is the map: what is available, what each one lets you do, and how to pick without guessing.

Principle: the honest starting point

"Which model should I use?" has no answer until you say what you want out of it. Most bad modeling starts with someone skipping that sentence.

Every model is the same sentence

$$y = f(x_1, x_2, \ldots, x_p) + \text{error}$$

- y is the thing you want to know. The x's are what you already have.
- f is the part the model supplies. Choosing a model is choosing what shapes f is allowed to take.
- The error is everything f does not account for. Some models make a claim about it. Others say nothing at all.

Principle: that is the whole menu

Every family in this unit is a different answer to two questions. What is f allowed to look like, and do we say anything about the error?

What this unit gives you

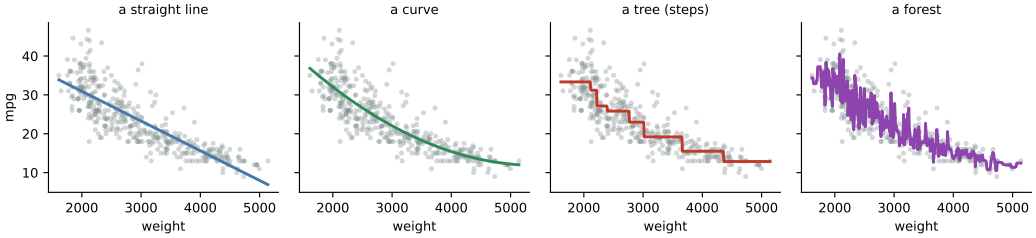
Things to know

- the main model families, and what each one assumes
- the difference between a probability model and an algorithmic one
- the five jobs people hand a model, and how they conflict
- which operations a model supports, and which it does not
- how sample size and the goal narrow the choice

Mistakes to stop making

- picking the algorithm before naming the goal
- asking a random forest for a p-value
- comparing an AIC against a cross-validation score
- reading a penalized coefficient as an effect
- reaching for flexibility on a few hundred rows

What f can look like



- **A straight line.** One slope, and you can say it out loud in units.
- **A curve.** One extra term, still readable.
- **A tree.** A step function, which is a set of if-then rules.
- **A forest.** An average of hundreds of trees, and here it is chasing every bump.

Same 392 cars and the same predictor in all four. Only f changed.

Two kinds of thing get called a model

A probability model

Says where the data came from. It writes down a distribution for the outcome:

$$y \sim \text{Normal}(b_0 + b_1 x,\ \sigma^2)$$

Everything else follows from that sentence.

An algorithmic model

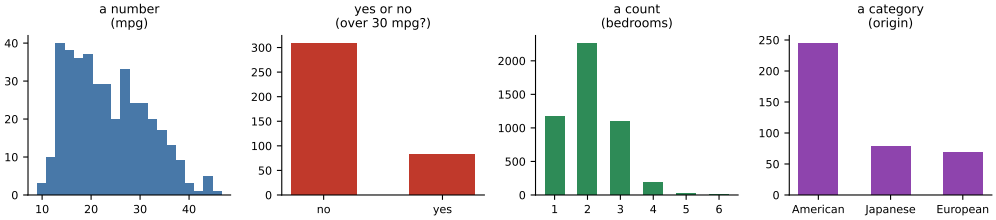
Says nothing about where the data came from. It gives you a rule that turns inputs into a prediction, tuned until the predictions are good.

A random forest is an average of 500 trees. That is the whole claim.

Principle: this split decides what you are allowed to ask

A distribution gives you a likelihood, and a likelihood gives you standard errors, p-values, confidence intervals, and AIC. With no distribution there is no likelihood, and none of those exist. You get uncertainty by resampling or by holding data out instead.

Start with the type of y



This is the one part of the choice that is not a judgment call, and it removes most of the menu before you have thought about anything else.

What each type of y points to

y is	Example	Probability model	Algorithmic
a number	mpg, rent, revenue	linear regression	trees, forests, boosting
yes or no	churn, default, click	logistic regression	classification trees and forests
a count or rate	tickets per hour	Poisson, negative binomial	boosting on counts
a category	which of four plans	multinomial logistic	classification models
time to an event	months until churn	survival models	specialized versions

Trap: forcing the wrong type

Running an ordinary linear regression on a yes/no outcome will produce numbers. Some of them will be predicted probabilities above 1 or below 0, which should tell you what happened.

The probability models

Model	Outcome it is for	What it says
Linear regression	a number	the mean moves in a straight line, with normal scatter around it
Logistic regression	yes or no	the log-odds move in a straight line
Poisson regression	a count, or a rate	counts arrive at a rate that depends on the inputs
Multinomial	one of several categories	the same idea with more than two outcomes
Mixed models	grouped or repeated data	rows inside a group are related, and the model says how
Bayesian versions	any of the above	the same models, plus a prior, giving a distribution over the answer

All of these are the same move: name a distribution, let the inputs shift its center.

The algorithmic models

Model	How it works	What it is good at
Decision tree	splits the data on one variable at a time	a rule a human can follow, and read out loud
Random forest	averages hundreds of trees grown on resampled data	solid predictions with little tuning
Gradient boosting	each tree fixes the last one's mistakes	usually the best raw accuracy on tabular data
k-nearest neighbours	predicts from the most similar past cases	simple, and surprisingly hard to beat locally
Neural networks	layers of weighted sums, fit by gradient descent	images, text, and very large datasets

None of these assume a distribution, so none of them hand you a p-value.

The awkward middle: penalized regression

- **Lasso** and **ridge** look like linear regression with a penalty added for large coefficients.
- The form is a probability model, so people report the coefficients as if the usual machinery applied.
- It does not. The penalty deliberately biases every coefficient toward zero, so the printed standard errors and p-values are not describing what they claim to.

Trap: a coefficient that has been shrunk is not an effect

Use these to *screen* many candidates down to a few, then refit an ordinary regression on the survivors if you need to defend a number. Report the screening step when you do.

Five jobs people hand a model

1. **Predict.** Give me a number for this new case.
2. **Explain.** Does X actually move Y, and by how much?
3. **Screen.** Out of these 200 columns, which few are worth tracking?
4. **Decide.** Given what it costs to be wrong each way, what should we do?
5. **Deploy.** Run this every night for two years, and be able to explain any single output to someone who is unhappy about it.

Principle: say which one out loud first

These are five different jobs. A model that is excellent at the first can be useless at the second, and the choice that makes the fifth easy usually costs you accuracy on the first.

The jobs, with the models attached

Job	What you would reach for	Why that one
Predict a number	boosting or a forest with plenty of rows, regression when rows are scarce	judged only on out-of-sample error, so nothing else constrains it
Predict yes or no	logistic regression if anyone must explain it, boosting if not	you need a probability first, then somebody picks a threshold
Explain one effect	linear or logistic regression	only a probability model gives a coefficient you can defend
Screen many columns	the lasso	shrinks most coefficients to zero and leaves a short list
Decide under cost	anything that outputs a calibrated probability, plus a cost table	the threshold is a business choice, not a modeling one
Deploy	the simplest model that clears the bar	fewer moving parts, and every output can be explained

The jobs pull against each other

Job	What it needs	What it will cost you
Predict	flexibility, and lots of rows	coefficients nobody can read
Explain	a simple form and honest standard errors	accuracy you could have had
Screen	a method that handles many columns at once	valid p-values on the survivors
Decide	calibrated probabilities and a cost table	extra work nobody asked for
Deploy	stability, few moving parts, a clear story	the last few points of accuracy

In practice: a strong thing to say

"Before I pick anything: is this going in a report, or in production? The answer changes what I would build."

Your turn #1: which model, and why

Your turn: name the type of y first, then the family

1. A hospital wants to know which of 60 measurements taken at intake are worth continuing to collect.
2. A subscription business wants, for each customer, the chance they cancel next month.
3. A city wants to know whether a new bus lane reduced average commute time.
4. A support team wants to know how many tickets will arrive each hour tomorrow.

Two of these have the same type of y and call for different models anyway. Find them.

Your turn #2: answer

1. **Screening, many columns.** The lasso. y is whatever outcome they care about, but the job is cutting 60 down to a handful, and that is what a penalty is for. Report it as a shortlist, not as effects.
2. **Yes or no.** Logistic regression, or boosting if nobody needs to read it. Note that the ask is a *probability*, not a label, and somebody still has to choose the cutoff for acting on it.
3. **A number, but the real problem is causal.** Linear regression with the bus lane as the predictor of interest. The model is the easy part. Whether the lane was the only thing that changed is Unit 6.
4. **A count.** Poisson regression. Forcing hourly ticket counts into an ordinary linear model will happily predict negative tickets at 3 a.m.

Principle:

Items 2 and 3 share nothing. Items 1 and 3 could both end up as a regression, for completely different reasons.

The verbs

Every model gets some of these done to it. Which ones are available is not the same for all of them.

- **Estimate parameters.** Find the slopes, the splits, the weights.
- **Tune hyperparameters.** Settings you choose rather than estimate: tree depth, the lasso penalty, k in nearest neighbours.
- **Predict.** Push a new case through and get a number back.
- **Test.** Ask whether a coefficient could plausibly be zero.
- **Put an interval on it.** Around a coefficient, or around a prediction.

Principle: parameters are learned, hyperparameters are chosen

A parameter comes out of fitting. A hyperparameter you set beforehand, and the honest way to set it is by cross-validation, never by trying values until the answer looks right.

Which verbs each family supports

	Probability model	**Algorithmic model**
Parameters mean something	yes, a slope is an effect	no, splits are bookkeeping
Standard errors and p-values	free from the likelihood	none exist
Confidence interval for an effect	yes	not really
Prediction interval	from the assumed distribution	from held-out errors
AIC or BIC	yes	undefined, no likelihood
Out-of-sample error	yes	yes, and it is the only option
Hyperparameters to tune	few	many, and they matter

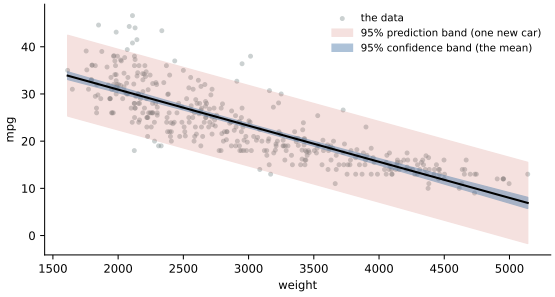
The right-hand column is shorter. That is the trade you make for the accuracy.

What "going deep" on one model actually involves

Once you commit to a family, the work is the same list every time. Unit 3 walks all of it for linear regression, and later units repeat the pattern for the others.

1. **Fit it**, and read the parameters in the units of the problem.
2. **Check the form.** Look at what the model missed. Is the shape you assumed roughly the shape that is there?
3. **Decide what belongs in it**, and be able to say why each variable is there.
4. **Tune what is not estimated**, on folds you fixed in advance.
5. **Put uncertainty on both things**: the parameters, and a prediction for a new case.
6. **Ask the actual question.** Usually that is a test or an interval on one coefficient, not the model as a whole.
7. **Score it on data you did not fit**, once.
8. **Decide whether to ship it**, and whether you can explain any single output.

Two intervals that get confused



Confidence band. Around the *mean*. Where is the true line? It shrinks toward nothing as n grows.

Prediction band. Around *one new car*. It stays wide forever, because individual cars genuinely differ.

Trap: quoting the wrong one

"We predict \$420k, 95% CI \$418k to \$422k" is a confidence interval passed off as a prediction. The customer is asking about one house, and that range is far wider.

Simple or complicated

	Reach for simple	**Reach for flexible**
Goal	explaining, screening, deploying	predicting
Rows	hundreds	tens of thousands and up
Columns	few, chosen for a reason	many, and you do not know which
Shape	you can say what it should look like	you have no idea, and want the fit to find it
Audience	someone who will challenge a number	someone who only sees the output

Principle: flexibility is bought with sample size

A complicated model is a bet that you have enough data to pin down everything you just let vary. On 300 rows that bet usually loses.

Comparing models by a number

- **AIC and BIC** trade fit against the number of parameters. They need a likelihood, so they only apply to probability models, and only when the models are fit to the same rows. Lower is better, and the raw value means nothing on its own.
- **Out-of-sample error** fits on part of the data and scores on the rest. It works for every model on this list, including the ones with no likelihood.
- Cross-validation is that idea done five or ten times over so the score is less dependent on which rows you happened to hold out.

Trap: comparing scores that are not comparable

An AIC and a cross-validated error are different quantities on different scales. Pick one method and use it for every candidate, on identical data.

Three choices, worked

Situation	y **is**	**What I would fit, and why**
1,100 loan applications, 12 clean columns, a regulator reviews every rejection	yes or no	Penalized logistic to pick the columns, then refit an ordinary logistic on the survivors so the coefficients are defensible. Say that you screened first.
6 million ad impressions, 400 columns, only click rate matters	yes or no	Gradient boosting. Same type of y as above and a completely different answer, because nobody will ever read a coefficient.
900 stores, weekly complaint counts, did the new policy reduce them	a count	Poisson regression with a policy indicator, and an offset if the stores differ in size. Then ask whether anything else changed the same week.

The first two have identical outcome types. The audience decided them, not the data.

The two-model habit

- Fit the simplest defensible model and a flexible one. Both, every time.
- If they agree, report the simple one. It is easier to defend, cheaper to maintain, and less likely to break when the data shifts.
- If they disagree, you have learned something specific: there is structure the simple model is missing. Go find out what it is.

In practice: a strong thing to say

"The forest beat the regression by four points, so I looked for what it was using, found the threshold at 30 days, added that to the regression, and it caught up. I shipped the regression."

Your turn #2

Your turn: name the family and the reason

1. 1,100 loan applications, 12 clean columns, and a regulator who will ask you to justify every rejection.
2. 6 million ad impressions, 400 columns, and nobody will ever read a coefficient.
3. 60 trial patients, one treatment variable, and the question is whether the drug works.
4. 25,000 sensor readings where you know the relationship bends but not how.

The family is the easy half. The reason is the half that gets graded.

Your turn #2: answer

1. **Logistic regression.** Not because it predicts best, but because "justify every rejection" means somebody will read a coefficient, and only a probability model gives you one you can defend.
2. **Boosting or a forest.** Six million rows makes flexibility affordable and nobody needs the parameters to mean anything.
3. **A t-test, or a regression with the treatment indicator.** With $n = 60$ and one variable of interest, a flexible model would spend your data estimating things nobody asked about. Randomization is doing the work here, not the model.
4. **Both.** A flexible fit to see the shape, then name it and put it in a regression you can explain.

Principle:

Three of these four were decided before anyone looked at the data.

LLM check: mistake or not?

LLM check: you asked for help choosing a model

"With 200 patients and 45 predictors I'd recommend a random forest. It handles nonlinearity and interactions automatically and gives you feature importances, so you can report the top predictors as the key risk factors and their p-values."

Three separate problems. Find them before the next slide.

LLM check: answer

- **The ratio.** 200 rows against 45 columns is the corner where flexible models fit well and predict badly. A penalized regression is the honest choice.
- **"Automatically" costs something.** Discovering interactions means spending data to find them. With 200 rows you cannot afford the search.
- **A forest has no p-values.** There is no likelihood, so the quantity being asked for does not exist. And an importance score ranks correlated columns close to arbitrarily, so it is not an effect either.

Principle:

The answer is not wrong about what a forest does. It is wrong about whether this dataset can pay for one, and about what the output means.

Choosing a model, in order

1. **Name the job.** Predict, explain, screen, decide, or deploy.
2. **Name the outcome type.** A number, a yes or no, a count, a category. This is not a judgment call and it removes most of the menu.
3. **Decide whether you need parameters that mean something.** If yes, you are in the probability column, and that is the end of the discussion.
4. **Count what you have against what you are estimating.** Rows, columns, and noise.
5. **Fit the simple one first.** It is your baseline and often your answer.
6. **Fit a flexible one on the same folds**, compare with one method, and take the simpler model when they are close.
7. **Report on data you touched once**, and say what else you tried.

The traps, all in one place

Trap: the ones that bite most often

- Choosing the algorithm before naming the job.
- Asking an algorithmic model for a p-value or an AIC.
- Reading a penalized or shrunk coefficient as an effect.
- Comparing an in-sample criterion against an out-of-sample one.
- Tuning a hyperparameter until the answer looks right.
- Quoting a confidence interval when the question was about one new case.
- Reaching for flexibility you do not have the rows to pay for.

What you should be able to do with software

Nobody is asking you to memorize syntax. You are expected to know which procedure the question calls for, what to hand it, and what the output means.

You should be able to	**What you would reach for**
Fit a linear or logistic regression	`smf.ols` / `smf.logit`
Read the coefficient table	`fit.summary()`
Fit a forest or a boosted model	`RandomForestRegressor`, `HistGradientBoosting...`
Tune a hyperparameter honestly	`GridSearchCV` over folds you fixed first
Compare probability models	`fit.aic`, on identical rows
Compare anything at all	`cross_val_score` with one shared `KFold`
Screen many columns	`LassoCV`, then read what survived

If you cannot say what the output means, producing it was worth nothing.

Where we go next

- You now have the map. The rest of Part I walks the parts of it that matter most.
- **Unit 3** takes the linear model seriously: reading a slope, what happens when you add a variable, and how to tell whether the fit is real.
- Later units come back to the right-hand column, to prediction intervals, and to the categorical outcomes this map only names.

Principle: the through-line

The model is a means. The job you were given decides which one, and saying that job out loud is most of the work.